

Cerberus - How Radix achieves infinite linear scalability while preserving atomic composability

Substate

6



This is a substate.

A substate is a unit of data that represents a change to the state of the ledger. It does not represent the state itself.

Kind of like entries in a set of accounts, you have to add up all the entries to get your balance.

So in this example there are four substates. Adding them together gives you the state, which in this case is 8.

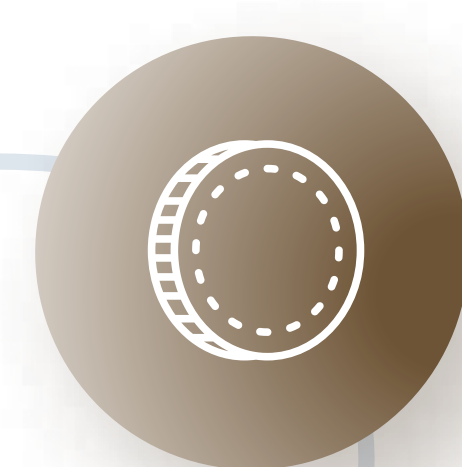
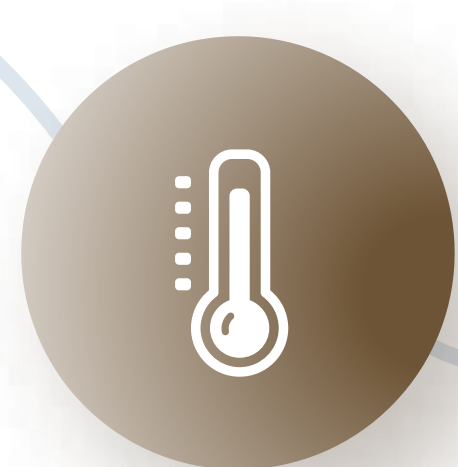
The state of the entire Radix ledger will be composed of millions upon millions of substates scattered across millions upon millions of shards.



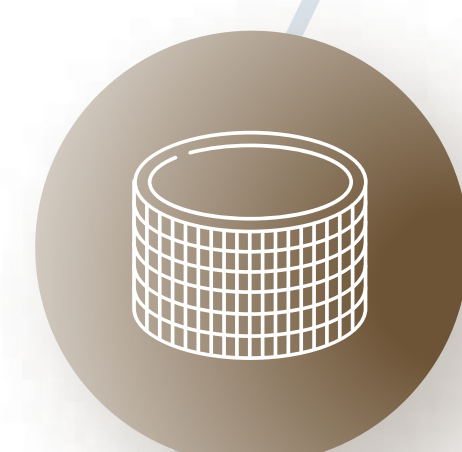
Once a substate is created, it is there forever and cannot be changed - it's immutable. To alter the effect of a substate on the ledger, you would have to create a new substate.

Additionally, a substate doesn't have to just represent a number. It can represent *anything*.

From something as simple as the average temperature on a certain day.

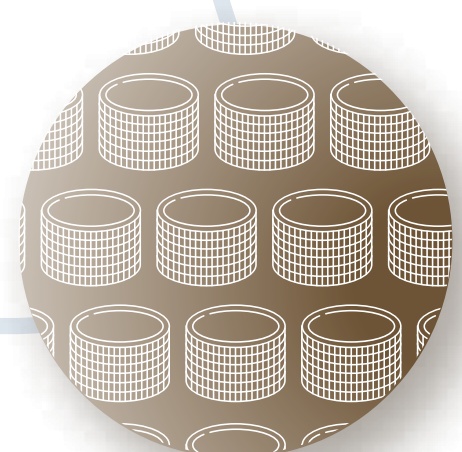


To the ownership of a single token in a single address.



Or multiple tokens in a single address.

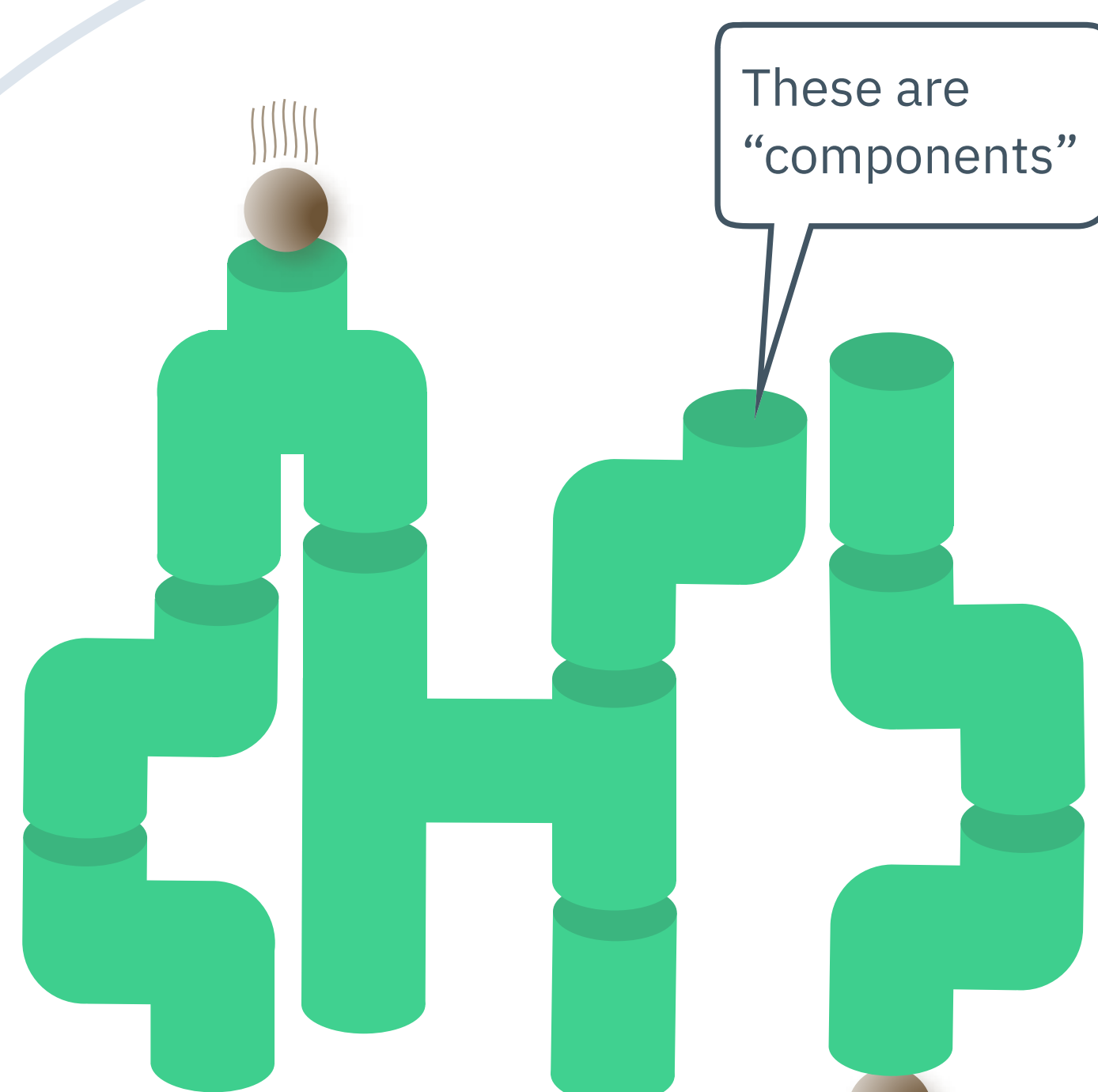
Or be as complex as the deployment of self-executing computer code - smart contract "components" - on-ledger.



Or thousands of different tokens across thousands of addresses.

A substate can therefore be as simple or as complex as needed - from a single token, to an entire dApp *and* all of its users' tokens, anything can be contained in a single substate, as a substate is just data.

However, Cerberus can't tell the difference between a substate that represents the temperature, or a substate that represents a token - Cerberus only helps nodes come to agreement about which substates should be created and become part of the ledger.



Radix Engine

The part of Radix that helps nodes make sense of what a substate represents is Radix's application layer, the Radix Engine.

The Radix Engine allows developers to create "components", which is code (similar to a smart contract) that defines rules that tells nodes what to do with certain substates. These components can be snapped together in any way shape or form to create decentralized applications "dApps".

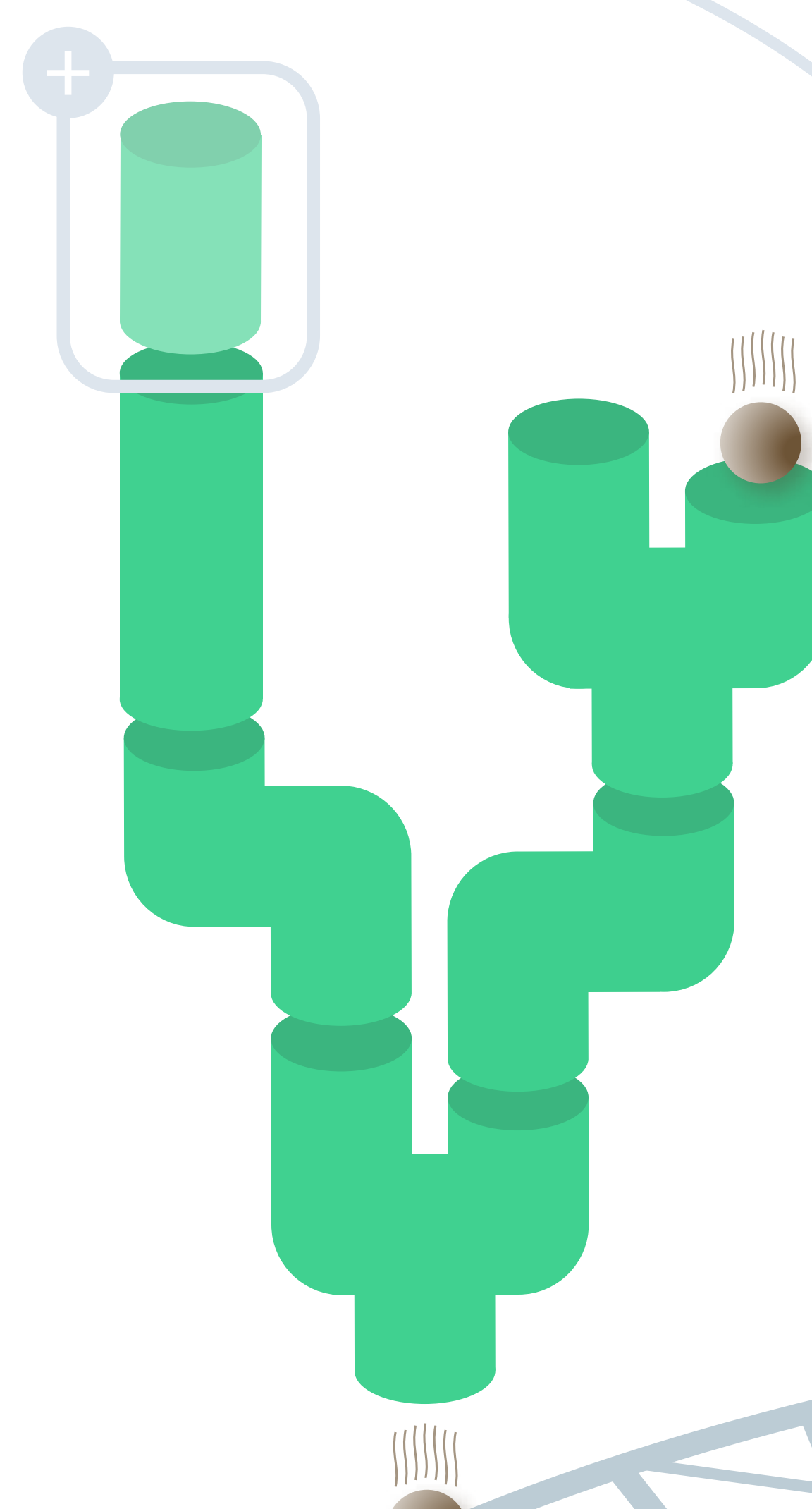
So for example, the rules that govern a substate that represents the temperature - for example in a decentralized agricultural derivatives application - will be different from the rules that govern the substate that represents a token.

Because Radix is decentralized and permissionless anyone can build their own component. Because components are open-source, anyone can iterate on anyone else's.

A process akin to natural selection will result in the best components having the most liquidity and use. With time, countless layers of components will be built, each extending and innovating on what came before, and they will all be atomically composable.

Radix is thus a decentralized self-perpetuating set of rules that grows and evolves with time, and every rule can interact seamlessly with every other rule.

A cornerstone of Radix is to reward those that make it better. That's why automated royalties are baked into the heart of the Radix Engine, so that every time a component gets used, the developer who created that component gets rewarded.



But now back to Cerberus - how is a substate created?

The version of Cerberus described in this infographic series is scheduled to launch as part of the fully sharded Radix Xi'an release. Please visit www.radixdlt.com for details on the Radix roadmap.